Optimization Similarity Fuzzy Inference for Traffic Signal Control

Case study report: an isolated four-way intersection

Name: Ch Pranav
Roll No: 1601-24-748-099
Semester / Section: 5th Semester, CSM2
Sustainable Development Goal addressed: SDG 11, Sustainable Cities and Communities

Abstract

Signalized intersections are among the biggest bottlenecks in urban road networks. Fixed-time and simple actuated controllers cannot react well when traffic fluctuates, and machine-learning controllers are often opaque and computationally heavy. Fuzzy logic offers an interpretable middle path, but classical Mamdani and Sugeno controllers use fixed rules and membership functions.

This report studies the Optimization Similarity Fuzzy Inference (OSFI) technique applied to traffic signal control at an isolated four-way intersection, based on Esmaeili, Anjomshoae, Shahsavari-Pour, Srisurin and Banomyong (2025), published in *Multimodal Transportation*. The paper reports that OSFI lowers average vehicle delay against both Mamdani and Sugeno controllers in every phase of both a moderate-traffic and a heavy-traffic scenario, with reductions of up to 5.73%.

The report also documents a small Python simulation prototype written to illustrate the workflow. The prototype is an educational approximation, not an exact reproduction of the paper. In the prototype, the OSFI-inspired controller clearly beats fixed-time control but does not beat the Sugeno-style controller, and this report states that result openly.

1. Introduction

More than half of the world's population now lives in cities, which puts heavy pressure on road infrastructure. Signalized intersections interrupt traffic flow and create queues and delay. Re-timing signals is a high-impact and cost-effective way to reduce congestion because it does not need new road space.

This project looks at how a fuzzy-logic signal controller, improved with a similarity-optimization step (OSFI), can reduce delay at a single four-way intersection while staying interpretable and cheap to compute.

1.1 Relevance to Sustainable Development Goal 11

This case study targets **SDG 11: Sustainable Cities and Communities**. Smarter signal control reduces congestion and delay at urban intersections without building new roads. Lower delay and shorter queues mean less time spent idling in traffic, which supports more efficient and liveable cities.

2. Problem Statement

Existing signal-control approaches fall short in the following ways.

Approach	Limitation
Fixed-time / pre-timed	Preset cycles ignore real demand, so delays grow when traffic fluctuates
Actuated / semi-actuated	Reacts to detectors, but the logic remains relatively rigid
Mamdani and Sugeno fuzzy	Interpretable, but fixed rules and membership functions limit real-time adaptability
Machine learning / deep reinforcement learning	Adaptive, but black-box, computationally heavy, and harder to verify and trust

Research gap. Fuzzy logic is widely used for isolated intersections, yet the OSFI technique had not been rigorously applied to traffic signal control.

3. Objectives

1. Apply OSFI to signal timing and phase sequencing.
2. Beat Mamdani and Sugeno controllers on average delay.
3. Keep fuzzy logic's interpretability and low compute cost.
4. Implement a simple simulation prototype to demonstrate the workflow and compare controllers.

4. Reference Paper

Item	Detail
Authors	Esmaeili, Anjomshoae, Shahsavari-Pour, Srisurin and Banomyong
Year	2025
Journal	Multimodal Transportation
DOI	10.1016/j.multra.2025.100234

The presentation that this report is based on is titled *Optimization Similarity Fuzzy Inference for Traffic Signal Control*.

5. Methodology (as described in the paper)

5.1 Case study setup

The case study is an isolated four-way intersection with approaches North, East, South and West.

- There are 4 signal phases, with one green at a time. Each phase serves through traffic and right turns.
- Each approach is modelled as an **M/M/1 queue**: Poisson arrivals at rate λ (vehicles per 10 seconds), a single server (the green phase), and first-in-first-out discipline.
- Two scenarios are tested:
  - **Moderate traffic:** arrival rate Medium, queue Long.
  - **Heavy traffic:** arrival rate High, queue Very Long.
- The OSFI controller is compared against Mamdani and Sugeno benchmarks, all using centroid defuzzification.

5.2 The OSFI idea

OSFI is an incremental refinement of fuzzy reasoning. It stays rule-based, interpretable and lightweight. It works in three steps.

1. **Fuzzy rule base:** IF arrival rate and queue length THEN green extension.
2. **Truth table:** ideal input-output pairs define the desired outcome.

3. **Maximize similarity:** tune the rules so the actual outputs match the benchmark truth table.

The optimization objective is written in the presentation as:

maximize Φ(E*B) = Sim(EA → EB , E*A → E*B)

In words, the tuned rule outputs should be as similar as possible to the ideal input-output behaviour.

5.3 Controller architecture

The controller has three modules.

Module	Full name	Input	Output
NPS	Next Phase Selector	Queue lengths of the 4 phases	The phase with the longest queue
GPE	Green Phase Extender	Arrival rate and queue lengths	Green-time extension (computed with OSFI)
SPM	Switch Phase Module	NPS phase and GPE extension	Executes the phase switch

5.4 Inside the Green Phase Extender

The GPE uses two-stage fuzzy inference.

- **FC1 (16 rules).** Inputs are arrival rate (Zero, Low, Medium, High) and queue length (Short, Medium, Long, Very Long, over 0 to 60 m). The output is Extension1.
- **FC2 (13 rules).** Inputs are Extension1 and the queue lengths of the other three phases. The output is Extension2, the final green extension (Zero, Short, Medium, Long).
- The AND operation uses the Łukasiewicz implicator, the output is a weighted average, and defuzzification uses the centroid method.

6. Results Reported in the Paper

Average delay is in seconds. Lower is better. "Vehicles exiting" is the number of vehicles discharged in each phase.

6.1 Moderate traffic

Phase	Mamdani delay	Sugeno delay	OSFI delay	Mamdani exits	Sugeno exits	OSFI exits
East	1.048	1.018	1.011	328	320	310
South	1.210	1.199	1.182	398	397	387
West	1.172	1.183	1.153	165	145	138
North	1.062	1.074	1.022	173	175	173

OSFI reduces delay in every phase: by 1.62% to 3.78% against Mamdani and by 0.69% to 4.84% against Sugeno.

6.2 Heavy traffic

Phase	Mamdani delay	Sugeno delay	OSFI delay	Mamdani exits	Sugeno exits	OSFI exits
East	1.137	1.126	1.098	347	321	298
South	1.178	1.201	1.165	381	385	362
West	1.172	1.154	1.132	152	137	128
North	1.144	1.131	1.082	186	185	178

Delay reductions are 1.11% to 5.73% against Mamdani and 1.94% to 4.52% against Sugeno, with the largest gain on the North phase.

6.3 What the numbers say

- **16 of 16** delay comparisons favour OSFI (4 phases × 2 scenarios × 2 baselines).
- The largest delay reduction is **5.73%** (heavy traffic, North phase, against Mamdani).
- **Caution:** exit counts are lower for OSFI in most phases. The paper reports the differences as gains and, in its own phase analysis, notes lower throughput in East and West. The delay gains therefore should not be read as an unqualified improvement.

7. Implementation: Python Simulation Prototype

7.1 Purpose and scope

The file `traffic_simulation.py` is a time-stepped simulation of a four-way intersection. It compares four controllers: **Fixed-time**, **Mamdani-style fuzzy**, **Sugeno-style fuzzy** and an **OSFI-inspired similarity-tuned fuzzy** controller.

The code states clearly that it is **not an exact reproduction** of the paper. The presentation does not give the full membership-function parameters, the rules, or the original truth table, so the prototype uses transparent, illustrative definitions instead. It is meant to demonstrate the workflow and the code structure, not to reproduce the published numbers.

7.2 Tools

The prototype uses Python with NumPy (random numbers and array maths), Matplotlib (graphs), and the standard `csv` and `pathlib` modules for output.

7.3 Simulation parameters

Parameter	Value	Meaning
SIM_TIME	3600 s	Total simulated time (one hour)
WARMUP	300 s	Initial period excluded from metrics
MIN_GREEN / MAX_GREEN	8 s / 35 s	Bounds on green duration
BASE_GREEN	10 s	Base green time before extension is added
CLEARANCE	2 s	Simplified all-red time between phases
VEHICLES_PER_SECOND	0.90	Mean discharge capacity during green
SEED	42	Base random seed

Arrival rates in vehicles per second for the approaches (North, East, South, West):

Scenario	North	East	South	West
Moderate	0.10	0.12	0.09	0.11

Scenario	North	East	South	West
Heavy	0.18	0.20	0.16	0.19

7.4 How the simulation works

1. **Arrivals.** Each second, every approach receives a Poisson-distributed number of new vehicles. Each vehicle is stored in a queue with its arrival time.
2. **Phase selection.** When the current green and clearance period are finished, the controller chooses the next phase. Fixed-time simply rotates through the phases. All adaptive controllers choose the approach with the longest queue (this plays the role of the Next Phase Selector).
3. **Green duration.** The chosen controller computes a green extension from the queue length and arrival rate. Green time is `BASE_GREEN + extension`, limited between `MIN_GREEN` and `MAX_GREEN`. Fixed-time always uses 15 s.
4. **Discharge.** During green, vehicles leave the active queue at a random rate with mean 0.90 vehicles per second. Waiting time is recorded as exit time minus arrival time, but only after the warm-up period.
5. **Clearance.** After each green, a 2 second clearance period passes before the next phase starts.
6. **Metrics.** The run reports average delay, throughput (vehicles discharged after warm-up), average queue per approach, total arrivals and vehicles still queued at the end.

7.5 The four controllers

Fixed-time. Rotates through the phases in order with a constant 15 s green. It is the non-adaptive baseline.

Mamdani-style fuzzy. Queue length and arrival rate are converted to Low, Medium and High membership degrees using simple linear and triangular functions. Six illustrative rules combine them with the minimum operator (AND). Each rule points to an output extension of 0, 3, 3, 7, 15 or 11 seconds. The clipped triangular output shapes are combined, and the extension is the **centroid** of the combined shape.

Sugeno-style fuzzy. It uses the same membership functions and the same six rules, but the output is a **weighted average** of the crisp rule outputs. This avoids the sampling step and is cheaper to compute.

OSFI-inspired similarity-tuned fuzzy. The prototype shows the OSFI workflow in a simplified form.

- A transparent **target table** defines the desired extension from a demand score, `0.75 × queue + 24 × arrival rate`: 0 s below 6, 4 s below 14, 9 s below 24, and 15 s otherwise. This stands in for the paper's truth table.
- A compact lookup table is built over 6 queue bins (0, 5, 12, 22, 35, 50) and 5 arrival-rate bins (0.10 to 0.50). Each cell stores the target output.
- At run time, the controller picks the **nearest cell** to the current queue length and arrival rate and returns that stored extension.

7.6 Outputs

Running `python traffic_simulation.py` prints a summary table, writes `results.csv`, and saves four graphs in a `graphs` folder: `average_delay.png`, `throughput.png`, `queue_length.png` and `queue_over_time.png`.

8. Results of the Simulation Prototype

The prototype was run once with the settings above. Delay is in seconds. Throughput is the number of vehicles discharged after warm-up. Average queue is vehicles per approach.

8.1 Moderate traffic

Controller	Average delay (s)	Throughput	Average queue	Arrivals	Unfinished
Fixed-time	25.89	1367	2.69	1497	8
Mamdani	22.98	1384	2.41	1519	8
Sugeno	**20.23**	1428	**2.19**	1548	8
OSFI-inspired	24.03	**1458**	2.65	1564	**4**

8.2 Heavy traffic

Controller	Average delay (s)	Throughput	Average queue	Arrivals	Unfinished
Fixed-time	56.23	2415	10.34	2649	38
Mamdani	43.22	**2429**	7.95	2640	**18**

Controller	Average delay (s)	Throughput	Average queue	Arrivals	Unfinished
Sugeno	**36.60**	2402	**6.66**	2626	21
OSFI-inspired	38.58	2382	6.95	2613	27

8.3 Delay of the OSFI-inspired controller compared with the others

A positive value means OSFI-inspired had lower delay.

Comparison	Moderate	Heavy
vs Fixed-time	+7.2%	+31.4%
vs Mamdani	-4.6%	+10.7%
vs Sugeno	-18.8%	-5.4%

traffic_simulation.py output (results.csv) · one run per controller, 3,600 s

Figure 1. Delay, throughput and average queue for each controller. The throughput panel is not a like-for-like comparison, because the arrival streams differ slightly between controllers (see section 9.3).

9. Discussion

9.1 What the prototype shows

- All three fuzzy controllers improve on fixed-time control. The gain is small under moderate traffic and large under heavy traffic, where the OSFI-inspired controller cuts delay by about 31% against fixed-time.
- Under heavy traffic the OSFI-inspired controller also beats the Mamdani-style controller by about 11%.
- The Sugeno-style controller gives the lowest delay in both scenarios, so the prototype does **not** reproduce the paper's headline result that OSFI beats both Mamdani and Sugeno.

9.2 Why the prototype differs from the paper

The difference is expected, and it comes from known simplifications.

1. **No real similarity optimization.** The OSFI-inspired table copies a heuristic target table directly. It does not run an optimization that maximizes a similarity measure Φ against a truth table, and the target table itself is invented for demonstration.
2. **Unused part of the table.** The arrival-rate bins run from 0.10 to 0.50, while the simulated arrival rates are only about 0.09 to 0.20. Most of the arrival dimension is never used, which makes the lookup coarse.
3. **Coarse nearest-cell lookup.** The OSFI-inspired controller jumps between a few fixed extension values, while the Mamdani and Sugeno controllers vary smoothly with the inputs.
4. **Constant arrival input.** The fuzzy controllers receive the true mean arrival rate of each approach, not a measured value, so the arrival input does not change during a run.
5. **Single-stage inference.** The paper uses a two-stage GPE (16 rules, then 13 rules) with an extension that also considers the other three queues. The prototype uses one stage and six rules.
6. **Different models and scales.** The paper uses an M/M/1 model with arrivals per 10 s and reports delays near 1 second. The prototype uses a one-second time step with explicit clearance time, so its delays are tens of seconds. The absolute numbers cannot be compared with the paper.

9.3 Fairness of the comparison

The code comment says the same seed is used for comparable arrival streams, but the seed actually changes with the controller index. The total arrivals therefore differ slightly between controllers (for example 1497 to 1564 in the moderate scenario). Throughput figures in particular are not directly comparable, and a single run per controller is too little to claim a winner.

9.4 Throughput and delay

The paper's own results show lower exit counts for OSFI in most phases, and this should be kept in mind when reading the delay improvements. Delay and throughput need to be reported together.

10. Limitations

- Single intersection and simulation only, with no field data.
- Fixed membership functions.
- Vehicles only, with no pedestrians, cyclists or transit.
- Needs accurate real-time sensing of queues and arrivals.
- Similarity optimization can be computationally heavy.
- The prototype is an approximation and uses one run per setting.

11. Future Work

- Multi-intersection and network-wide control.
- Self-tuning membership functions and rules.
- Field pilots and real-world validation.
- Multimodal and extreme-demand scenarios.
- Comparison with type-2 fuzzy and neuro-fuzzy controllers.
- For the prototype: implement a true similarity-maximization step, use the paper's exact rules and membership functions, share one arrival stream across all controllers, and repeat each run with 10 to 30 seeds, reporting the mean and standard deviation.

12. Conclusion

1. According to the paper, OSFI beats Mamdani and Sugeno on average delay in every tested case.
2. It keeps the interpretable, rule-based and low-compute character of fuzzy logic.
3. The gains are incremental, and broader validation is still needed. The work is a first step, covering an isolated intersection in simulation, with networks and field trials as the next stage.

The Python prototype demonstrates the workflow of fuzzy signal control with fixed-time, Mamdani, Sugeno and OSFI-inspired controllers. It confirms that adaptive fuzzy control clearly outperforms fixed-time control, especially under heavy traffic. It does not reproduce the paper's claim that OSFI is best, which is consistent with the prototype's simplified, non-optimized implementation. The case study supports SDG 11 by showing how better signal control can reduce urban congestion.

References

1. Esmaeili, Anjomshoae, Shahsavari-Pour, Srisurin and Banomyong (2025). Optimization Similarity Fuzzy Inference for Traffic Signal Control. *Multimodal Transportation*. doi:10.1016/j.multra.2025.100234.
2. Source presentation: *Optimization Similarity Fuzzy Inference for Traffic Signal Control* (case study: an isolated four-way intersection).
3. Source code: `traffic_simulation.py`, OSFI-inspired traffic signal simulation prototype.

Appendix A: How to Run the Prototype

1. Install the requirements: `pip install numpy matplotlib`.
2. Run `python traffic_simulation.py` from a folder where files can be written.
3. Read the printed summary, open `results.csv`, and view the PNG graphs in the `graphs` folder.

Appendix B: Files Produced by the Prototype

File	Content
results.csv	Scenario, controller, average delay, throughput, average queue, arrivals and unfinished vehicles
graphs/average_delay.png	Average delay by controller and scenario
graphs/throughput.png	Vehicles discharged after warm-up
graphs/queue_length.png	Average queue per approach
graphs/queue_over_time.png	Queue lengths over time, OSFI-inspired controller, heavy traffic